RectifiedTraj Revisited: An Expanded Evaluation of Map-Free Causal GPS Trajectory Denoising*

Dexin Ren
dexinren@usc.edu
University of Southern California
Los Angeles, CA, USA

Shang-Ling Hsu
hsushang@usc.edu
University of Southern California
Los Angeles, CA, USA

Chen Chu
chenchu@usc.edu
University of Southern California
Los Angeles, CA, USA

Lifeng Lin
linlifeng@g.ecc.u-tokyo.ac.jp
University of Tokyo
Tokyo, Japan

Ryosuke Shibasaki
shiba@reitaku.jp
Reitaku University
Kashiwa, Japan

Noboru Koshizuka
noboru@iii.u-tokyo.ac.jp
University of Tokyo
Tokyo, Japan

Cyrus Shahabi
shahabi@usc.edu
University of Southern California
Los Angeles, CA, USA

Abstract

Global Positioning System (GPS) observations from mobile devices are often corrupted by signal reflection, weak reception, and environment-dependent error, reducing the reliability of location-based applications. A practical mobile denoiser must correct GPS data as it arrives, avoid dependence on map or network services, and be lightweight enough for on-device execution. These needs motivate **causal**, **map-free**, and **lightweight** GPS denoising.

Our previous work introduced RectifiedTraj, a causal, map-free, and lightweight denoiser that uses rectified flow for one-step trajectory correction. In this study, we revisit RectifiedTraj through a substantially broader evaluation and analysis. We compare it with classical filters and smoothers, map matching, direct regression, residual regression, a causal multilayer perceptron (MLP), and diffusion. We analyze prediction stability as an input segment (chunk) covers a larger geographic range and examine errors caused by limited preceding context at causal chunk starts. This motivates evaluating how removing error-prone predictions near chunk starts trades accuracy against throughput.

Across the expanded comparisons using the NUMOSIM, Patterns of Life, and BlogWatcher datasets, RectifiedTraj remains the strongest overall denoiser. It remains stable when longer sampling intervals cause each input segment to span a larger geographic distance, while clean-state reconstruction baselines develop rapidly growing errors. A same-backbone residual-regression baseline performs comparably to RectifiedTraj. The expanded analyses identify correction-field prediction as the main source of stability. Pointwise analysis also identifies elevated errors at causal chunk starts and shows that discarding these early predictions can reduce them. Finally, we evaluate all three RectifiedTraj architectures directly on a Google Pixel 4a; every model completes inference within approximately 124 ms.

CCS Concepts

• **Information systems** → **Spatial-temporal systems**; • **Computing methodologies** → *Machine learning approaches*; • **Human-centered computing** → *Ubiquitous and mobile computing systems and tools.*

Keywords

GPS trajectory denoising, rectified flow, causal modeling, map-free denoising, mobile sensing

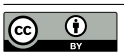
ACM Reference Format:
Dexin Ren, Shang-Ling Hsu, Chen Chu, Lifeng Lin, Ryosuke Shibasaki, Noboru Koshizuka, and Cyrus Shahabi. 2026. RectifiedTraj Revisited: An Expanded Evaluation of Map-Free Causal GPS Trajectory Denoising. In *The 2nd ACM SIGSPATIAL International Workshop on Urban Mobility Foundation Models (UMFM '26), November 03–06, 2026, Riverside, CA, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3849729.3856129

1 Introduction

Mobile applications receive Global Positioning System (GPS) positions as a stream of points. These positions support navigation, geofencing, fitness tracking, mobility analysis, and activity recognition [3, 29, 30]. However, GPS measurements from mobile devices can be shifted by weak satellite signals, reflections from buildings, and changes in the surrounding environment [19, 28]. An inaccurate position can also produce an incorrect speed, heading, travelled distance, or stop event. The error is therefore passed from the raw trajectory to the applications that use it.

Real-world GPS observations are generated and streamed in time order. When the current observation must be denoised, future observations have not yet been collected and therefore cannot be used. Many downstream applications, including navigation, also require corrected positions in real time. Causal online denoising

*Code and artifacts: https://github.com/DexinRen/RectifiedTraj.



This work is licensed under a Creative Commons Attribution 4.0 International License.
UMFM '26, Riverside, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3055-9/2026/11
https://doi.org/10.1145/3849729.3856129

is therefore a natural way to correct GPS signals as they arrive, using only the current and previous observations. Mobile operation also takes place under changing network conditions. An Internet connection or suitable road-map data may not always be available, and a correction service that depends on them may stop working when they are lost. Map-free correction therefore offers a more independent solution that is less tightly coupled to network connectivity, map providers, or external services. The problem studied in this paper is how to reduce GPS coordinate error causally, using only the observed trajectory, while keeping the correction practical on a mobile device.

This correction problem often occurs on mobile devices. Their compact, low-power positioning hardware and antennas can further limit signal quality [19], while the denoising model must share the device's limited computing and energy resources with other applications [10, 18]. Causal and map-free operation are therefore not sufficient by themselves. A practical GPS denoiser should also be lightweight enough to process incoming observations on a mobile-device central processing unit (CPU).

Together, these constraints define three practical requirements for mobile GPS denoising: **causal** operation, **map-free** correction, and **lightweight** execution. In our previous research, we introduced RectifiedTraj, a GPS denoiser based on rectified flow [13, 22]. The model predicts a correction field from the current and previous observations and applies that field in one inference step. That study established the feasibility of this approach through its main accuracy comparison and a single-core CPU evaluation.

In this study, we revisit RectifiedTraj with a broader evaluation of the same central problem. The main trajectory-error metrics remain the primary evidence because direct reduction of coordinate error is the purpose of the model. We extend the comparison to additional learned methods, including same-backbone direct- and residual-regression baselines, a causal multilayer perceptron (MLP), and a diffusion-based denoiser, and include Meili as a practical map-matching reference. We also examine how prediction error changes across the causal context instead of reporting only one average for an entire trajectory. This analysis is important because a low overall error can hide poor correction of the newest observation, which is the point needed by an online application. Finally, we complement the CPU results reported in the short paper with measurements on a Google Pixel 4a, allowing the runtime and resource cost to be evaluated on a real mobile phone.

The main contributions are as follows:

- **Broader evaluation:** We extend RectifiedTraj beyond the original causal-filter comparison to global smoothers, learned denoisers, map matching, strict online evaluation, and real-world transfer.
- **Failure-mode analysis:** We show that direct regression becomes unstable as the within-chunk spatial range grows. A same-backbone residual-regression baseline remains stable and performs comparably to RectifiedTraj, identifying correction-field prediction as the main source of stability. We separately characterize causal startup error and the accuracy–throughput trade-off introduced by head-buckle reassembly.
- **Real-device deployment:** We demonstrate CPU-only inference on a Google Pixel 4a.

2 Related Work

GPS correction methods include causal filters, global smoothers, map matching, and learning-based denoisers. Classical filters and smoothers are efficient and easy to interpret [5, 8, 21, 23], map matching provides road-consistent paths that are useful for navigation [17, 26], and learned models can represent more complex noise patterns [4, 9, 12, 14]. These methods are designed for different settings, and their suitability depends on the information, infrastructure, and computing resources available when correction is performed. The following subsections review these method families and the baselines evaluated in this paper.

Classical Filtering and Smoothing. Alpha-Beta filtering, Kalman filtering, and one-sided Hampel filtering can process observations causally and are efficient enough for online use [5, 7, 8]. Classical filters also have a practical implementation advantage in embedded navigation systems. Kalman filters are commonly implemented in navigation firmware, and some commercial global navigation satellite system receivers execute tightly coupled Kalman filtering directly on the receiver chip [20]. This supports fast, real-time estimation without placing the full filtering workload on the device's main processor. The Rauch–Tung–Striebel (RTS) and centered Savitzky–Golay smoothers are widely used for denoising completed trajectories, where both past and future observations are available to estimate each corrected position [21, 23]. Causal filters and global smoothers therefore use different amounts of temporal context: causal filters operate as observations arrive, while global smoothers can use the completed trajectory. Both are widely used in the settings for which they were designed and provide established reference points for online and offline GPS correction.

Map Matching. Map matching estimates the road segment or route that most likely explains an observed GPS trace. It is widely used in navigation because it produces a path that is consistent with the road network [15, 17, 31]. Valhalla Meili snaps observations to positions on road-network edges [26]. These positions commonly lie on the mapped road centerline and therefore may not represent the user's actual coordinates. For conventional navigation and route guidance, however, representing the user on the road centerline is generally sufficient. Map matching is also an important downstream application of GPS denoising. Providing timely and more accurate GPS observations can help a map-matching system identify the correct road segment, determine whether the user remains on the planned route, and detect when the user has moved off that route. Thus, GPS denoising and map matching serve different but connected purposes: denoising improves the coordinates supplied to the map matcher, while map matching converts those observations into a road-constrained path. Meili therefore provides a practical reference for the correction behavior commonly seen in everyday navigation applications. Its road-constrained output represents map matching rather than causal coordinate denoising.

Learning-based and Generative Denoising. Learning-based denoisers use neural models to learn a direct relationship between noisy and cleaner trajectories [4, 9, 12, 14]. Their main advantage is the ability to fit complex, data-dependent noise patterns, which can provide better denoising quality when the training data represent the conditions encountered during deployment. The deployment

cost of a learned denoiser depends on both its model size and its inference procedure. Some learned models rely on graphics processing unit (GPU) acceleration to achieve practical inference speed, which can be difficult to provide on mobile devices with limited computing power, memory, and energy.

Direct clean-coordinate regression offers a simple one-step prediction objective. For online denoising, its network can be made causal so that each corrected point uses only the current and previous observations. Diffusion models refine their predictions through multiple denoising steps, allowing them to represent flexible correction processes [2, 6, 16, 32, 33]. Because every step requires another model evaluation, the selected step count creates a direct trade-off between refinement and inference latency. Rectified flow learns a more direct transport path and can support one-step correction [11, 13]. It offers another design point that combines learned modeling of complex noise with low-delay inference.

Together, these methods offer complementary forms of GPS correction: causal filtering for streaming estimates, global smoothing for completed trajectories, map matching for road-constrained paths, and learned models for data-dependent noise. When selecting a method for real-world deployment, its denoising quality must be considered alongside causal operation, infrastructure dependence, and inference cost. The next section explains how the causal, map-free, and lightweight setting shapes RectifiedTraj's design.

3 Methodology

Real-world GPS trajectories vary in length and arrive as time-ordered streams. Feeding a complete observed trajectory to the model would therefore require it to handle sequences of varying and potentially unbounded length. We divide each observed trajectory into fixed-size windows, providing the model with a standardized input and bounded per-window processing cost. To support lightweight execution, we choose a window size of 256 points. We represent each window as $X_\tau = \langle x_{\tau,i} \rangle_{i=1}^{256}$ and refer to it as a chunk, where the subscript $\tau \in [0, 1]$ indicates its noise level and x_i represents the i-th individual GPS point. When $\tau = 0$, X_0 is the clean ground-truth chunk; when $\tau = 1$, X_1 is the corresponding raw input chunk.

3.1 Chunking and Reassembly

Fixed-size chunks keep the model input bounded, but they introduce a causal boundary effect. In strict causal and online prediction, the correction for a point x_i can depend only on x_i and the points preceding it. A point near the beginning of a chunk therefore has less preceding context than a point later in the chunk, making the beginning more prone to prediction error. We quantify this effect later through the pointwise prediction-error analysis.

To reduce this boundary effect, we introduce a Q-point overlap between adjacent chunks: each non-initial chunk begins with the final Q retained points from the preceding chunk. We call these initial positions the head-buckle region; the remaining $256 - Q$ positions form the retained payload. The first chunk forms its head-buckle region by repeating its first point. If the final payload is incomplete, its last point is repeated to fill the window and the added positions are marked as padding.

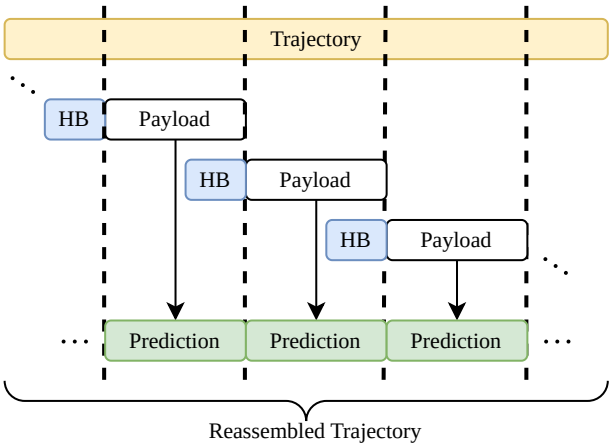


Figure 1: Fixed-size chunking and reassembly. HB denotes the head-buckle region whose predictions are discarded; the retained payload predictions form the reassembled trajectory.

During reassembly, predictions from the head-buckle region are discarded, padding is removed, and the retained payload predictions are concatenated. This prevents the error-prone startup predictions from entering the reconstructed trajectory while preserving every original trajectory position. Figure 1 illustrates this overlap and prediction-assembly process.

3.2 Rectified-Flow Denoising

Once a trajectory has been divided into chunks, we must carefully define what the model predicts. It is natural to predict the clean chunk X_0 directly. However, the magnitude of the clean-coordinate target can change greatly as the trajectory progresses, while the displacement required to correct each raw point may remain small. The scale of a direct coordinate prediction is therefore affected by the point's position along the trajectory and is not bounded by the GPS-error magnitude. This mismatch can produce a large training loss and make direct prediction numerically difficult. To address this problem, we introduce rectified flow (RF). Rather than reproducing every clean coordinate, the model learns the displacement between each raw point and its clean counterpart. RF represents this displacement as a velocity field:

$$\mathbf{V} = X_1 - X_0.$$

Because X_1 and X_0 are represented in the same local coordinate frame, their shared trajectory displacement is removed by the subtraction. The prediction target therefore describes the GPS correction itself, and its magnitude depends on the raw-to-clean error rather than the point's distance from the beginning of the chunk. We examine this numerical effect experimentally by comparing direct clean-coordinate prediction and rectified-flow correction as the spatial range within a chunk grows.

RF connects the clean and raw chunks through a straight interpolation path [13]:

$$X_\tau = (1 - \tau)X_0 + \tau X_1 = X_0 + \tau \mathbf{V}, \qquad \tau \in [0, 1].$$

Under this straight-path assumption, the ideal velocity remains constant along the path:

$$\frac{dX_\tau}{d\tau} = X_1 - X_0 = \mathbf{V}.$$

A denoising network $v_\theta(X_\tau, \tau)$ receives a chunk at noise level τ and estimates this velocity field:

$$v_\theta(X_\tau, \tau) \approx \mathbf{V}.$$

Because the ideal velocity is constant along the straight path, the same target $\mathbf{V}$ applies at every noise level. The network therefore learns the displacement separating the raw and clean chunks instead of reproducing the full clean-trajectory coordinates.

At inference, the observed raw chunk is X_1 at $\tau = 1$. RectifiedTraj predicts its correction field once and subtracts it from the raw chunk:

$$\widehat{X}_0 = X_1 - v_\theta(X_1, 1).$$

This direct update is the main reason RF fits online GPS denoising. Its simple straight-path data hypothesis supports one-step denoising. Requiring only one model evaluation keeps inference inexpensive and fast, providing the low-latency correction needed by real-world applications that process GPS observations as they arrive.

3.3 Causal Local–Global Denoising Network

After defining the prediction objective, the next design question is what causal context the network should use to estimate the velocity field. A GPS point viewed alone provides little evidence for distinguishing measurement error from genuine movement. Nearby observations reveal short-range displacements and changes in direction, while the longer history of preceding points provides a broader view of the trajectory's movement pattern. Accurate correction can therefore benefit from both local and longer-range context.

To use these two forms of context, RectifiedTraj employs a Hybrid network that combines a causal one-dimensional convolutional neural network (CNN) with a causal Transformer. The CNN first processes neighboring observations to extract local movement features. These features are merged with a pointwise embedding of the input chunk before entering the Transformer. The Transformer can therefore use both the extracted local patterns and the input coordinates while examining the longer history of preceding points [27].

The network receives the interpolated chunk X_τ and its noise level τ as input. The CNN uses left-padded causal convolutions, the Transformer uses causal attention, and the final layer-normalization operation and linear projection are applied independently to each position. All components therefore preserve causality, and the resulting velocity prediction is causal.

To examine whether both local and longer-range context contribute to denoising, we additionally train a causal CNN-only model and a causal Transformer-only model alongside the Hybrid architecture. Their performance comparison is reported as an architecture ablation in the Experiments section.

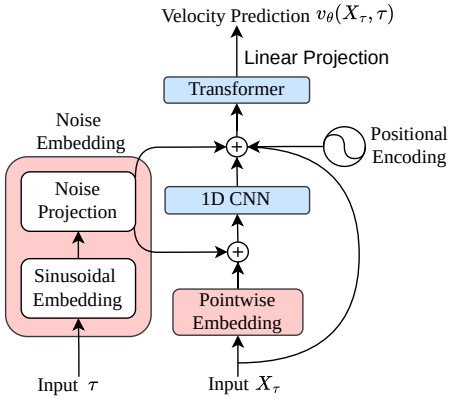


Figure 2: Causal Hybrid architecture, reproduced from our previous study [22]. The CNN extracts local movement patterns, and its features are combined with the input-chunk embedding before entering the causal Transformer. A pointwise linear projection maps each Transformer output to its rectified-flow velocity prediction.

3.4 Learning the Velocity Field

Because the straight-path data hypothesis of rectified flow supports one-step inference and avoids the numerical difficulty of direct coordinate prediction, RectifiedTraj can be trained with a simple velocity-matching objective [13]. For each paired clean and raw chunk, we sample a noise level τ, construct the corresponding intermediate chunk X_τ, and train the network to recover the velocity field defined in Section 3.2. Sampling different values of τ provides varied training inputs while keeping the correction target unchanged.

For each valid, non-padded point, we compare the predicted velocity with the target velocity using their squared Euclidean distance. Let $\mathcal{I}$ denote the set of non-padded point indices in a chunk. The loss averages the pointwise errors over these valid positions and across the training examples:

$$\mathcal{L}(\theta) = \mathbb{E}_{X_0, X_1, \tau}\left[\frac{1}{|\mathcal{I}|}\sum_{i \in \mathcal{I}} \|v_\theta(X_\tau, \tau)_i - \mathbf{V}_i\|_2^2\right].$$

Restricting the sum to $\mathcal{I}$ prevents artificial padding from contributing to the training objective.

4 Experiments

Our evaluation is organized around five questions. First, how accurately does RectifiedTraj correct trajectories across sampling intervals, spatial domains, and noise conditions? Second, how does its one-step causal correction compare with classical filtering and smoothing, direct and residual regression, diffusion, and practical map matching? Third, how does prediction stability change as the spatial range covered within a trajectory chunk increases? Fourth, where does prediction error occur within a causal chunk, and how does the head-buckle size Q trade boundary-error reduction against retained throughput? Fifth, can RectifiedTraj provide low-latency inference on a physical mobile phone? We address these questions

Table 1: Native evaluation dataset statistics. Dist. is the pointwise distance from raw input data to the available reference. Sampling statistics describe the native evaluation split. Radius applies only to BlogWatcher accuracy evaluation. Here, p95 denotes the 95th percentile, and N/A denotes not applicable.

	NUMOSIM	PoL	BlogWatcher
Eval. type	Exact	Exact	Accuracy
Traj. tested	200	200	200
Points / traj.	5000	5000	5000
Sampling median (s)	10.0	5.0	10.0
Sampling mean (s)	11.16	5.0	116.44
Sampling p95 (s)	10.0	5.0	303.0
Dist. mean (m)	20.220	36.765	54.566
Dist. median (m)	18.336	34.486	16.774
Dist. p95 (m)	42.181	72.605	94.418
Radius mean (m)	N/A	N/A	42.640
Radius median (m)	N/A	N/A	14.250
Radius p95 (m)	N/A	N/A	70.000

using two simulated datasets with exact ground truth—NUMOSIM-Kanto and Patterns of Life (PoL)—and one real-world dataset, BlogWatcher. We additionally analyze pointwise prediction error and measure inference on a physical smartphone.

4.1 Experimental Setup

Datasets. We evaluate RectifiedTraj on two simulated datasets with exact ground truth—**NUMOSIM-Kanto** and Patterns of Life (**PoL**)—and on the real-world **BlogWatcher** dataset using its provided reference regions. We use the same preprocessing protocol as our previous study [22]. All learned models are trained solely on NUMOSIM-Kanto at its native 10-second interval. NUMOSIM-Kanto contains heteroscedastic Gaussian noise and missing-point events [24]. PoL contains Atlanta trajectories generated by a different simulator [1]; we inject temporally correlated Ornstein–Uhlenbeck (OU) noise [25], making it a transfer test across geography, simulator, sampling interval, and noise type. BlogWatcher is a non-public real-world dataset with irregular sampling and reference locations and radii rather than an exact trajectory. According to the provider metadata, its records were collected with user consent and processed using differential privacy and k-anonymization. Table 1 summarizes these datasets.

Sampling protocol and evaluation tasks. Mobile GPS observations can become sparse or irregular, so a denoiser should remain reliable beyond its training interval. We test increasingly sparse NUMOSIM data at 30 seconds, 1 minute, and 2 minutes; PoL additionally changes the geography, simulator, and noise type; and BlogWatcher provides naturally irregular real-world sampling. These settings test whether correction remains reliable as conditions move away from training.

To evaluate sustained streaming rather than isolated inputs, each test split contains up to 200 trajectories and 5,000 points per trajectory. The benchmark covers 4.0 million NUMOSIM, 4.5 million PoL, and 1.0 million BlogWatcher points. Model selection, baseline calibration, and head-buckle analysis use data separate from the test trajectories.

Metrics. For NUMOSIM and PoL, we measure the pointwise Cartesian ℓ_1 distance between each prediction and its exact ground-truth point (exact test). Mean reports the average error across all retained predictions in the main trajectory benchmark. Online evaluates the stricter streaming setting in which each arriving point is corrected as the newest point of a 256-point chunk; for RectifiedTraj, this corresponds to retaining only the final prediction from each chunk using $Q = 255$. Mean and Online are identical for causal baselines whose predictions are unaffected by chunk reassembly. The head-buckle analysis additionally reports p95 error to reveal high-error boundary regions that may be hidden by the mean.

Because BlogWatcher does not provide exact ground truth, we report the proportion of predictions that fall within the circle defined by the provider's reference location and radius (accuracy test). We report this accuracy over the complete test set and over increasingly strict subsets whose provided radius is at most 30m, 15m, 10m, or 5m. All metrics are calculated on fixed test splits.

Evaluation configuration. All learned models use the same training budget and checkpoint-selection protocol, while methods that require parameter calibration use a separate calibration split. The learned models process 256-point chunks. RectifiedTraj performs one-step inference with $\Delta\tau = 1$, whereas the denoising diffusion implicit model (DDIM) uses its configured 500-step denoising process.

The main trajectory benchmark uses $Q = 0$, retaining the complete reassembled prediction, including the error-prone beginning of each chunk. The strict Online evaluation uses $Q = 255$ for RectifiedTraj and retains only the newest prediction as the GPS stream advances. Intermediate Q values are evaluated only in the separate head-buckle analysis on the NUMOSIM-Kanto native calibration split. DDIM-Hybrid retains the causal Hybrid architecture. However, its Online results are not reported because evaluation with $Q = 255$ requires approximately 256 times more chunk evaluations than the main trajectory benchmark. Combined with DDIM's 500-step denoising process, this makes a complete Online evaluation computationally prohibitive in our implementation. Completing all nine evaluated sampling settings would require more than half a year of computation.

Baselines. Our previous study mainly compared RectifiedTraj with map-free classical causal filters, including Alpha-Beta, Kalman, and one-sided Hampel filtering, to provide a closely scoped comparison among methods with similar operating conditions and deployment goals. This study broadens the comparison to method families widely used in or closely related to GPS correction, including map matching, global trajectory smoothers, and learned denoisers represented by a causal multilayer perceptron and a diffusion model.

To examine how causal denoising compares with methods that can use a completed trajectory, we add symmetric Hampel filtering [5] and the global smoothers Kalman-RTS [21] and centered Savitzky–Golay [23]. To compare rectified flow with other learned

denoising approaches, we add a causal MLP that predicts corrections directly and DDIM using the Hybrid architecture for iterative diffusion-based denoising. DirectReg-Hybrid uses the same Hybrid architecture as RectifiedTraj's Hybrid variant but directly predicts X_0, providing a controlled comparison of the prediction objective. ResidualReg-Hybrid instead predicts the correction $X_0 - X_1$ directly from the raw chunk X_1, without interpolated training inputs or noise-level conditioning, and reconstructs $\widehat{X}_0$ by adding the predicted correction to X_1. This same-backbone control isolates whether rectified-flow training provides a benefit beyond correction-field prediction. We also include Valhalla Meili as a practical map-matching reference, connecting map-free coordinate correction with the road-constrained output commonly used in navigation applications. When Meili cannot match an input point, we retain its raw coordinate so that every method is evaluated on the same complete trajectory.

The expanded results combine the causal-filter and RectifiedTraj measurements reported in our previous study [22] with the newly evaluated baselines under the same benchmark protocol. Raw GPS is included as the uncorrected reference. The evaluated RectifiedTraj models are the causal CNN, Transformer, and Hybrid variants, denoted RT-CNN, RT-Trans., and RT-Hybrid, respectively; each contains approximately one million parameters. Methods requiring parameter calibration use separate calibration data rather than the test trajectories.

4.2 Denoising Performance

NUMOSIM exact denoising. Our previous study established RectifiedTraj's advantage over closely scoped causal filters [22]. Table 2 broadens this comparison by adding global smoothers, map matching, a causal MLP, direct regression, residual regression, and diffusion. RT-Hybrid and ResidualReg-Hybrid perform comparably: ResidualReg-Hybrid leads at 10s, while RT-Hybrid leads at 30s, 1min, and 2min.

Among the newly added baselines, ResidualReg-Hybrid is the strongest: its Mean error increases from 3.01m at 10s to 5.02m at 2min, closely tracking RT-Hybrid from 3.04m to 4.84m. The global smoothers improve the raw trajectory in several settings but do not reach the learned models' accuracy. Meili produces road-constrained outputs and does not reduce exact coordinate error on this dataset. DirectReg-Hybrid and DDIM-Hybrid deteriorate sharply as the sampling interval grows; we examine this behavior in the prediction-stability analysis. This same-backbone comparison identifies correction-field prediction as the main source of stability; rectified-flow training provides no consistent improvement.

Figure 3 extends the qualitative comparison from our previous study [22] on the NUMOSIM-Kanto city-center setting near Japan's Imperial Palace. The visual comparison follows the quantitative ranking: ResidualReg-Hybrid, RT-Hybrid, and RT-Trans. produce consistently thinner trajectory traces, reflecting their lower prediction error.

PoL cross-noise transfer. Our previous study reported RectifiedTraj's transfer from NUMOSIM-Kanto to the different geography, simulator, and noise process represented by PoL [22]. Table 3 extends that evaluation with additional method families and the strict Online test. At PoL's native 5s interval, DirectReg-Hybrid achieves the lowest Mean error at 17.74m, while RT-Trans. achieves the lowest available Online error at 11.79m. RT-Trans. achieves the lowest Mean error at 10s and 30s, while ResidualReg-Hybrid leads at 1min and 2min.

In the Online results, RT-Trans. leads at 5s and 10s, RT-Hybrid at 30s, and ResidualReg-Hybrid at 1min and 2min. Its comparable performance across both exact-reference datasets further shows that rectified-flow training provides no consistent improvement over residual regression. Unlike their behavior on NUMOSIM, DirectReg-Hybrid and DDIM-Hybrid do not exhibit extreme error growth on PoL. The prediction-stability analysis examines this difference using the spatial distance covered within each chunk.

BlogWatcher accuracy test. BlogWatcher provides a reference location with a radius for each point rather than exact point-level ground truth, so we perform an accuracy test where a point is counted as accurate when its prediction falls inside the circle centered at the provided reference location with the corresponding radius. Because BlogWatcher is non-public and can only be evaluated by its data custodian, ResidualReg-Hybrid could not be evaluated on this dataset within the study period. Raw data already achieves 44.63% overall accuracy, meaning 44.63% of raw points are inside their provided accuracy circles before denoising. Therefore, methods below raw accuracy degrade the raw data under this metric. Table 4 reports both accuracy and ΔAcc. relative to raw data within each tier. Several filters reduce accuracy relative to raw GPS, while all RectifiedTraj variants improve over raw data and the strongest baseline. RT-Hybrid achieves the best overall accuracy at 57.05%, a +12.42 percentage-point improvement over raw data. RT-Trans. is strongest on the ≤5m and ≤10m subsets, while RT-Hybrid remains strongest on the broader overall, ≤30m, and ≤15m metrics.

Meili shows a different trade-off across the two evaluation types. In the exact-reference tests, its road-centerline predictions do not reduce exact coordinate error. BlogWatcher, however, contains much longer and more irregular sampling intervals, with a mean of 116.44 seconds and a p95 of 303 seconds. Under these conditions, the road network provides a useful spatial guardrail: successfully matched observations are constrained to plausible road geometry, while unmatched observations retain their raw coordinates. Meili consequently reaches 50.80% overall accuracy and becomes the strongest non-RectifiedTraj baseline. At the strictest ≤5m tier, it also exceeds RT-CNN and RT-Hybrid, although RT-Trans. remains slightly stronger. This result does not imply that map matching guarantees precise coordinates; rather, it suggests that road constraints can limit uncontrolled correction error when the appropriate road is identified, providing stable accuracy under sparse and irregular observations. Because 53.16% of points use raw fallback, the reported result reflects the combination of successful road-constrained matches and unchanged raw observations.

4.3 Prediction Stability as Trajectory Range Grows

Our methodology argues that predicting clean coordinates exposes a model to a target range that grows with trajectory displacement. To measure this range, for each non-overlapping 256-point clean-reference chunk, we calculate the maximum geodesic distance from its starting point to any other point in the same chunk. We refer

Table 2: Expanded NUMOSIM-Kanto exact trajectory denoising benchmark. Mean and Online report pointwise Cartesian ℓ_1 error in meters. Mean summarizes the standard trajectory evaluation, while Online reports strict streaming evaluation. RectifiedTraj Online uses $Q = 255$ so each chunk contributes only its newest prediction. DDIM-Hybrid uses 500 denoising steps. Methods marked with † are causal; N/A denotes an unavailable or inapplicable result.

	10s (native)		30s		1min		2min	
Method	Mean ↓	Online ↓	Mean ↓	Online ↓	Mean ↓	Online ↓	Mean ↓	Online ↓
Raw trajectory	20.22	20.22	19.92	19.92	19.92	19.92	19.95	19.95
Alpha-Beta†	13.94	13.94	19.76	19.76	19.78	19.78	19.84	19.84
Causal Hampel†	18.91	18.91	22.03	22.03	25.54	25.54	33.98	33.98
Kalman Filter†	14.99	14.99	19.66	19.66	19.82	19.82	19.95	19.95
Hampel	18.84	N/A	18.59	N/A	18.66	N/A	18.82	N/A
Kalman-RTS	9.02	N/A	17.01	N/A	18.38	N/A	20.20	N/A
Savitzky–Golay	9.68	N/A	12.72	N/A	17.36	N/A	26.40	N/A
Valhalla Meili	30.34	N/A	29.40	N/A	29.07	N/A	29.18	N/A
Causal MLP†	5.42	5.42	7.19	7.19	9.68	9.68	12.35	12.35
DirectReg-Hybrid†	6.61	19.63	102.84	256.69	283.94	563.32	393.15	784.76
ResidualReg-Hybrid†	**3.01**	**2.07**	3.44	2.86	4.22	4.10	5.02	5.45
DDIM-Hybrid†	18.15	N/A	839.56	N/A	2566.16	N/A	3566.32	N/A
RT-CNN†	4.49	4.50	5.22	5.87	6.33	7.53	7.35	9.13
RT-Trans.†	3.26	2.55	3.96	3.92	5.05	5.68	6.08	7.45
RT-Hybrid†	3.04	2.12	**3.43**	**2.85**	**4.17**	**4.07**	**4.84**	**5.23**

to this value as the within-chunk distance. Table 5 reports its p95 value across the evaluated chunks. The NUMOSIM results illustrate the resulting relationship: from 10s to 2min, DirectReg-Hybrid's mean error rises from 6.61m to 393.15m and DDIM-Hybrid's rises from 18.15m to 3,566.32m, whereas ResidualReg-Hybrid increases only from 3.01m to 5.02m and RT-Hybrid from 3.04m to 4.84m.

The 0.00km p95 value for PoL at 5s is not a measurement error. It reflects PoL's activity structure: agents often remain stationary at points of interest for extended periods, so more than 95% of the native 5s chunks have zero within-chunk distance.

NUMOSIM's raw GPS error remains approximately 20m across all intervals, while its p95 within-chunk distance grows from 2.36km to 24.35km. DirectReg-Hybrid's median error grows only from 3.00m to 4.36m while its mean reaches 393.15m, showing that the degradation is driven by extreme failures. PoL provides evidence in the opposite direction: despite its unseen domain and noise type, its p95 within-chunk distance never exceeds 3.76km, and neither DirectReg-Hybrid nor DDIM-Hybrid exhibits the same error explosion. The instability therefore aligns more closely with within-chunk spatial range than with GPS-error magnitude or noise mismatch alone.

This pattern supports the correction-field objective. DirectReg-Hybrid, ResidualReg-Hybrid, and RT-Hybrid use the same backbone, but DirectReg-Hybrid predicts X_0, whereas the other two predict equivalent correction fields. DirectReg-Hybrid becomes unstable as within-chunk distance grows, while ResidualReg-Hybrid and RT-Hybrid remain stable. Their comparable performance identifies correction-field prediction, rather than rectified-flow training alone, as the source of this stability. DDIM-Hybrid also deteriorates because its denoising process must reconstruct the absolute clean state.

4.4 Head-Buckle Analysis

As discussed in the methodology, a causal model can only use the current point and the points preceding it. This creates a particular problem for architectures that rely on global context: near the beginning of a chunk, much of that context is unavailable. We examine this effect in Figure 4 by plotting the average prediction error at each of the 256 positions across the evaluated chunks. RT-Trans. and RT-Hybrid show a clear concentration of error near the beginning of the chunk. Both architectures use a causal Transformer to incorporate broader preceding context, so their predictions improve as more points become available. RT-CNN, in contrast, relies on local features extracted through a bounded convolutional receptive field and does not show the same startup-error pattern. These results associate the elevated beginning error with the limited global context available to causal Transformer-based models at chunk startup.

Head-Buckle Sweep. The head-buckle mechanism is designed to prevent these error-prone startup predictions from entering the reassembled trajectory. Increasing Q removes more startup predictions, but it also reduces the number of retained predictions produced by each chunk evaluation. We therefore sweep Q using RT-Hybrid on the NUMOSIM-Kanto native 10s calibration split and examine its effect on prediction error and throughput. The

Table 3: Expanded PoL exact trajectory denoising benchmark. Mean and Online report pointwise Cartesian ℓ_1 error in meters. Mean summarizes the standard trajectory evaluation, while Online reports strict streaming evaluation. RectifiedTraj Online uses $Q = 255$ so each chunk contributes only its newest prediction. DDIM-Hybrid uses 500 denoising steps. Methods marked with † are causal; N/A denotes an unavailable or inapplicable result.

	5s (native)		10s		30s		1min		2min	
Method	Mean ↓	Online ↓	Mean ↓	Online ↓	Mean ↓	Online ↓	Mean ↓	Online ↓	Mean ↓	Online ↓
Raw trajectory	36.77	36.77	37.24	37.24	37.19	37.19	37.21	37.21	37.23	37.23
Alpha-Beta†	36.71	36.71	37.24	37.24	37.11	37.11	37.03	37.03	36.96	36.96
Causal Hampel†	36.28	36.28	40.43	40.43	39.84	39.84	41.66	41.66	54.46	54.46
Kalman Filter†	37.04	37.04	37.65	37.65	37.32	37.32	36.85	36.85	36.80	36.80
Hampel	36.54	N/A	37.46	N/A	36.24	N/A	35.44	N/A	35.05	N/A
Kalman-RTS	34.92	N/A	37.40	N/A	34.87	N/A	32.82	N/A	33.35	N/A
Savitzky–Golay	34.75	N/A	36.58	N/A	31.88	N/A	29.53	N/A	33.39	N/A
Valhalla Meili	44.81	N/A	42.95	N/A	41.72	N/A	41.10	N/A	40.46	N/A
Causal MLP†	24.19	24.19	25.06	25.06	25.48	25.48	27.30	27.30	31.61	31.61
DirectReg-Hybrid†	**17.74**	12.53	18.98	20.25	20.15	25.30	22.46	33.82	30.14	60.03
ResidualReg-Hybrid†	27.14	24.40	22.67	19.84	12.83	10.97	**9.87**	**9.08**	**10.77**	**12.13**
DDIM-Hybrid†	22.29	N/A	26.33	N/A	28.89	N/A	32.07	N/A	41.16	N/A
RT-CNN†	26.52	25.96	24.37	24.58	18.06	19.52	15.49	18.09	17.12	22.27
RT-Trans.†	19.21	**11.79**	**16.24**	**12.82**	**12.22**	11.93	11.64	13.31	14.75	20.39
RT-Hybrid†	24.25	21.19	20.20	17.30	12.44	**10.66**	10.16	9.72	11.25	13.18

Table 4: BlogWatcher native accuracy test. Accuracy is the fraction of predicted points that fall within the radius around the provider reference location. ΔAcc. reports percentage-point difference from raw data within the same tier. Tiered columns evaluate subsets whose radius is at most 30m, 15m, 10m, and 5m. Methods marked with † are causal.

	All		≤30m		≤15m		≤10m		≤5m	
Method	Acc. ↑	ΔAcc. ↑	Acc. ↑	ΔAcc. ↑	Acc. ↑	ΔAcc. ↑	Acc. ↑	ΔAcc. ↑	Acc. ↑	ΔAcc. ↑
Raw data	44.63%	0.00	43.45%	0.00	42.82%	0.00	38.52%	0.00	36.42%	0.00
Alpha-Beta†	36.33%	-8.30	32.11%	-11.34	29.62%	-13.20	19.28%	-19.24	15.32%	-21.10
Causal Hampel†	37.90%	-6.73	37.20%	-6.25	36.36%	-6.46	32.47%	-6.05	30.17%	-6.25
Kalman Filter†	43.52%	-1.11	42.20%	-1.25	41.49%	-1.33	36.95%	-1.57	34.84%	-1.58
Hampel	45.46%	+0.83	44.07%	+0.62	43.37%	+0.55	38.77%	+0.25	36.44%	+0.02
Kalman-RTS	35.72%	-8.91	33.55%	-9.90	32.18%	-10.64	26.37%	-12.15	23.57%	-12.85
Savitzky–Golay	35.43%	-9.20	31.08%	-12.37	27.69%	-15.13	16.06%	-22.46	10.89%	-25.53
Valhalla Meili	50.80%	+6.17	48.90%	+5.45	48.16%	+5.34	44.47%	+5.95	44.24%	+7.82
Causal MLP†	45.18%	+0.55	42.83%	-0.62	40.87%	-1.95	30.57%	-7.95	27.77%	-8.65
DirectReg-Hybrid†	21.81%	-22.82	19.00%	-24.45	17.02%	-25.80	8.96%	-29.56	5.78%	-30.64
DDIM-Hybrid†	18.29%	-26.34	15.54%	-27.91	13.78%	-29.04	6.22%	-32.30	4.29%	-32.13
RT-CNN†	55.17%	+10.54	53.61%	+10.16	51.28%	+8.46	45.19%	+6.67	42.10%	+5.68
RT-Trans.†	55.51%	+10.88	54.22%	+10.77	52.13%	+9.31	**47.12%**	**+8.60**	**44.74%**	**+8.32**
RT-Hybrid†	**57.05%**	**+12.42**	**54.64%**	**+11.19**	**52.15%**	**+9.33**	45.40%	+6.88	42.30%	+5.88

calibration split is kept separate from the test split, and the primary architecture comparison continues to use $Q = 0$.

At $Q = 0$, all 256 predictions are retained. At the other extreme, $Q = 255$ retains only the final prediction, for which the model has

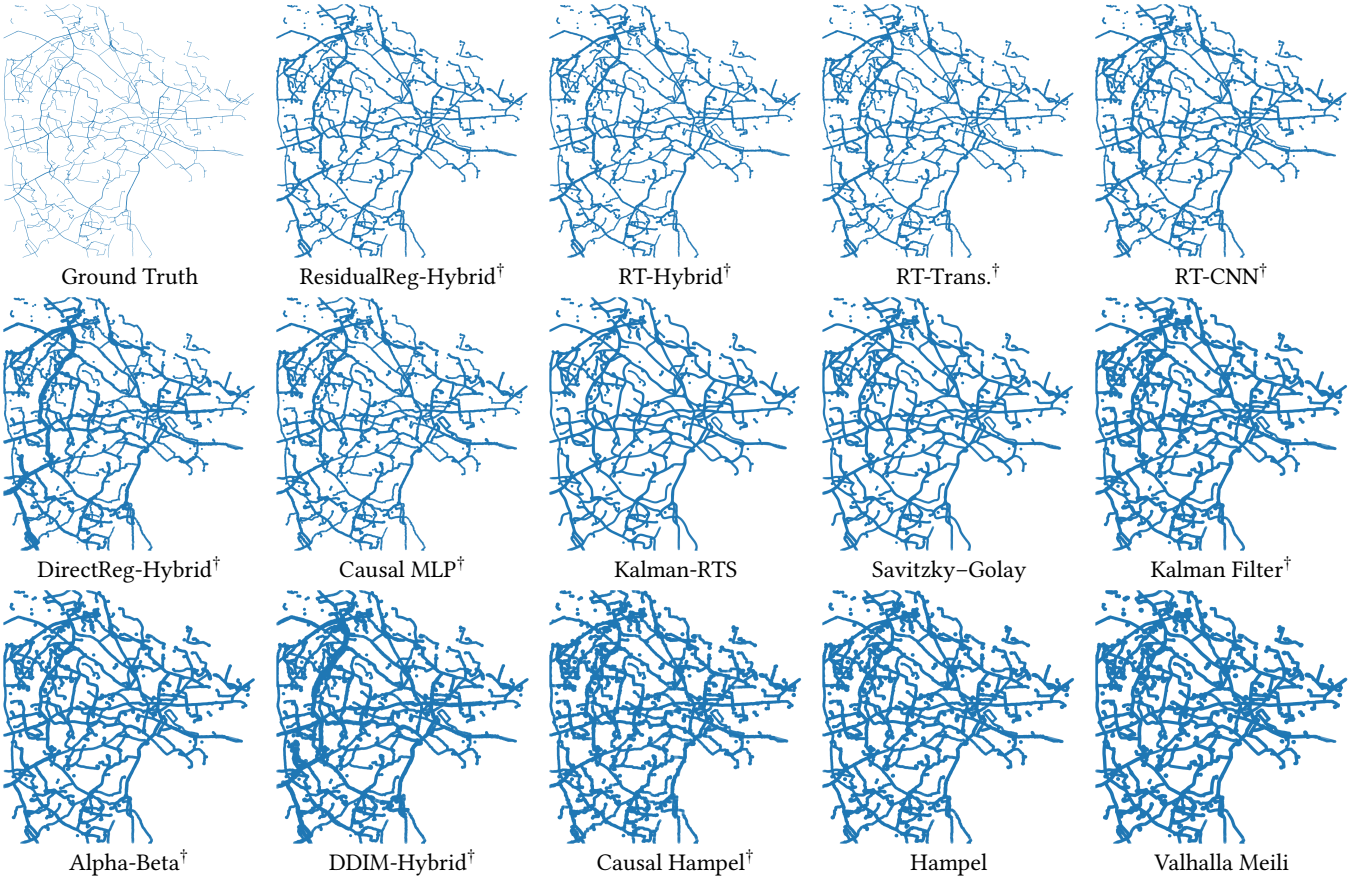


Figure 3: Qualitative trajectory comparison on the NUMOSIM-Kanto city-center subset, centered near Japan's Imperial Palace. Panels are sorted by average error. Each trajectory's line width encodes its average point error after global log normalization across all evaluated methods and trajectories; thinner trajectories indicate lower error. Methods marked with † are causal.

Table 5: p95 within-chunk distance in the exact-reference datasets, measured as the largest distance from the starting point to any other point in the same chunk.

Dataset	5s	10s	30s	1min	2min
NUMOSIM	N/A	2.36km	6.87km	16.34km	24.35km
PoL	0.00km	2.62km	2.95km	3.43km	3.76km

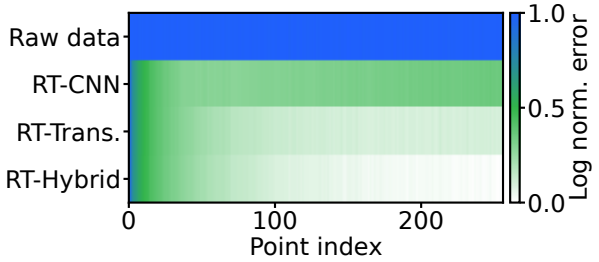


Figure 4: Chunk-level prediction error heatmap on the NUMOSIM-Kanto native 10s validation split. Each row represents the raw data or a RectifiedTraj variant, and each column represents one position in a 256-point chunk. Each cell reports the average pointwise error at that position across the evaluated chunks; darker colors indicate larger log-normalized error.

access to the complete preceding context within the chunk. The Mean and Tail errors are therefore identical at $Q = 255$.

Table 6 quantifies how removing different amounts of the startup region affects prediction accuracy and throughput. Increasing Q from 0 to 192 reduces Mean error from 3.04m to 2.16m and p95 error from 8.83m to 4.56m, while retaining a throughput of 9,769 retained predictions/s. In contrast, Tail error remains close to 2.1m throughout the sweep. This stable Tail error indicates that the improvement mainly comes from removing poorly contextualized startup predictions rather than changing predictions near the end of each chunk.

Larger Q values provide fewer retained predictions per model evaluation and consequently reduce throughput. At $Q = 255$, Mean

Table 6: Hybrid head-buckle sweep on the native NUMOSIM-Kanto 10s calibration split. Errors are pointwise Cartesian ℓ_1 distance (m); Tail is the last-point error, and throughput is retained predictions/s.

Q points	Mean ↓	Tail ↓	p95 ↓	retained/s ↑
0	3.04	2.10	8.83	**37,832**
16	2.68	2.11	6.48	35,599
32	2.55	2.16	5.77	33,232
48	2.44	2.11	5.36	31,078
80	2.34	**2.06**	4.94	26,699
128	2.24	2.06	4.66	19,361
192	2.16	2.06	**4.56**	9,769
240	2.13	2.09	4.60	3,244
255	**2.12**	2.12	4.63	249

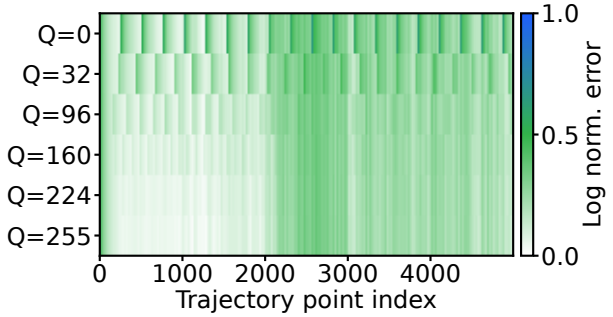


Figure 5: Log-normalized trajectory error for the RT-Hybrid head-buckle sweep on the NUMOSIM-Kanto native 10s calibration split. Rows show selected Q values; darker vertical bands indicate repeated high-error chunk boundaries after reassembly.

error reaches 2.12m, but throughput falls to 249 retained predictions/s. The head-buckle size therefore controls a direct trade-off between accuracy and throughput rather than having one universally optimal value.

Figure 5 shows periodic dark-green error strips at small Q, aligning with the error-prone chunk starts identified in Figure 4. These strips weaken as Q increases, visually confirming that head-buckle reassembly reduces repeated causal boundary artifacts.

4.5 Mobile-Device Inference

Our previous study demonstrated CPU-only feasibility using one Intel Core i7-10700K core [22]. We extend this evaluation to a 2020 Google Pixel 4a powered by a Snapdragon 730G processor. As shown in Table 7, all three RectifiedTraj architectures run without graphics acceleration or remote services; RT-Hybrid completes one-step correction in 124.235 ms, well below the shortest five-second sampling interval. DDIM has similar per-step latency but repeats it over 500 steps. Together, these results demonstrate that RectifiedTraj supports real-time inference on older mobile hardware.

Table 7: Model size and forward-pass latency on a Google Pixel 4a. Size is reported in megabytes (MB) and latency in milliseconds. The p90 value is the 90th-percentile latency. Throughput is input points processed per second; DDIM reports one denoising step.

Model	Size	Mean ↓	p90 ↓	input pts./s ↑
RT-CNN	4.20	11.020	12.435	23,230
RT-Trans.	4.26	53.818	59.443	4,757
RT-Hybrid	5.12	124.235	127.130	2,061
DDIM (one step)	5.12	120.958	128.966	2,116

5 Conclusion

Our previous study introduced RectifiedTraj as a causal, map-free, lightweight rectified-flow denoiser. Across sampling intervals, the expanded evaluation shows that it remains competitive with global smoothers, map matching, a causal MLP, direct regression, residual regression, and diffusion while correcting the current point without future observations or map access.

The expanded analyses identify correction-field prediction as the main source of stability. When each chunk spans more distance, DirectReg-Hybrid and DDIM-Hybrid errors grow sharply, whereas RectifiedTraj and ResidualReg-Hybrid remain stable by predicting corrections instead of absolute clean coordinates. Their comparable performance shows that rectified-flow training alone provides no consistent advantage over residual regression. Head-buckle reassembly further removes poorly contextualized chunk-start predictions, with Q controlling the accuracy–throughput trade-off. On a Pixel 4a, RT-Hybrid performs one-step correction in approximately 124 ms.

Limitations include simulated GPS noise, BlogWatcher's private circle-based references, inference measurements on one phone rather than a complete application, and no sampling interval below five seconds. Future work should evaluate denser mobile traces, more devices and environments, and real-world exact references. Within these limits, correction-field prediction provides a stable and computationally practical basis for causal, map-free GPS trajectory denoising.

Acknowledgments

This research has been funded in part by the NSF award DMS-2428039, and the Intelligence Advanced Research Projects Activity (IARPA) via the Department of Interior/Interior Business Center (DOI/IBC) contract number 140D0423C0033. The U.S. Government is authorized to reproduce and distribute reprints for the Governmental purposes, notwithstanding any copyright annotation thereon. Disclaimer: The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of NIH, NSF, IARPA, or the U.S. Government. The authors gratefully acknowledge the compute and data resources provided by Google Cloud Platform through the Google Cloud Research Credits program, as well as the unrestricted cash award from Google Research.

References

[1] Hossein Amiri, Will Kohn, Shiyang Ruan, Joon-Seok Kim, Hamdi Kavak, Andrew Crooks, Dieter Pfoser, Carola Wenk, and Andreas Züfle. 2024. The Patterns of Life Human Mobility Simulation. In *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems*. 653–656.

[2] Chen Chu, Hengcai Zhang, Peixiao Wang, and Feng Lu. 2024. Simulating human mobility with a trajectory generation framework based on diffusion model. *International Journal of Geographical Information Science* 38, 5 (2024), 847–878. arXiv:https://doi.org/10.1080/13658816.2024.2312199 doi:10.1080/13658816.2024.2312199

[3] Zhenni Feng and Yanmin Zhu. 2016. A survey on trajectory data mining: Techniques and applications. *Ieee Access* 4 (2016), 2056–2067.

[4] Kévin Cédric Guyard, Jonathan Bertolaccini, Stéphane Montavon, and Michel Deriaz. 2025. A Transformer Encoder Approach for Localization Reconstruction During GPS Outages from an IMU and GPS-Based Sensor. *Sensors* 25, 2 (2025), 522.

[5] Frank R. Hampel. 1974. The Influence Curve and Its Role in Robust Estimation. *J. Amer. Statist. Assoc.* 69, 346 (1974), 383–393. doi:10.1080/01621459.1974.10482962

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.

[7] Paul R. Kalata. 1984. The Tracking Index: A Generalized Parameter for Alpha-Beta and Alpha-Beta-Gamma Target Trackers. *IEEE Trans. Aerospace Electron. Systems* AES-20, 2 (1984), 174–182. doi:10.1109/TAES.1984.310438

[8] Rudolph Emil. Kalman. 1960. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering* 82, 1 (March 1960), 35–45. doi:10.1115/1.3662552

[9] Mosbeh R. Kaloop and Jong Wan Hu. 2015. Optimizing the De-Noise Neural Network Model for GPS Time-Series Monitoring of Structures. *Sensors* 15, 9 (2015), 24428–24444. doi:10.3390/s150924428

[10] Kaisen Lin, Aman Kansal, Dimitrios Lymberopoulos, and Feng Zhao. 2010. Energy-Accuracy Trade-off for Continuous Mobile Device Location. In *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services*. 285–298. doi:10.1145/1814433.1814462

[11] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2023. Flow Matching for Generative Modeling. In *11th International Conference on Learning Representations, ICLR 2023*.

[12] Sheng Liu, Weidong Xiang, and Vivekanandh Elangovan. 2023. A GPS Distance Error Forecast Model Based on IIR Filter Denoising and LSTM. *IEEE Transactions on Instrumentation and Measurement* 72 (2023), 1–12. doi:10.1109/TIM.2023.3284020

[13] Xingchao Liu, Chengyue Gong, and Qiang Liu. 2023. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *The Eleventh International Conference on Learning Representations (ICLR)*.

[14] Yonghui Liu, Qian Li, and Inhi Kim. 2025. Enhanced trajectory reconstruction from sparse and noisy GPS data: A progressive chunked transformer approach. *Communications in Transportation Research* 5 (2025), 100200.

[15] Yin Lou, Chengyang Zhang, Yu Zheng, and Xing Xie. 2009. Map-Matching for Low-Sampling-Rate GPS Trajectories. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 352–361. doi:10.1145/1653771.1653820

[16] Weibo Mao, Chenxin Xu, Qi Zhu, Siheng Chen, and Yanfeng Wang. 2023. Leapfrog Diffusion Model for Stochastic Trajectory Prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5517–5526.

[17] Paul Newson and John Krumm. 2009. Hidden Markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 336–343. doi:10.1145/1653771.1653818

[18] Jeongyeup Paek, Joongheon Kim, and Ramesh Govindan. 2010. Energy-efficient rate-adaptive GPS-based positioning for smartphones. In *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services*. 299–314. doi:10.1145/1814433.1814463

[19] Jacek Paziewski, Rafal Sieradzki, and Radoslaw Baryla. 2019. Signal characterization and assessment of code GNSS positioning with low-power consumption smartphones. *GPS Solutions* 23 (2019), 98. doi:10.1007/s10291-019-0892-5

[20] Uffe Pless, Alexander Somieski, Michael Ammann, and Carl Fenger. 2014. u-blox' 3D Dead Reckoning for Vehicle Applications. u-blox white paper, UBX-13005008. https://content.u-blox.com/sites/default/files/u-blox-3D-DeadReckoningForVehicleApps_WhitePaper_(UBX-14002187).pdf Accessed: 2026-08-29.

[21] H. E. Rauch, F. Tung, and C. T. Striebel. 1965. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal* 3, 8 (1965), 1445–1450. doi:10.2514/3.3166

[22] Dexin Ren, Shang-Ling Hsu, Chen Chu, Lifeng Lin, Ryosuke Shibasaki, Noboru Koshizuka, and Cyrus Shahabi. 2026. RectifiedTraj: Map-Free Causal GPS Trajectory Denoising with Rectified Flow. In *Proceedings of the 34th ACM International Conference on Advances in Geographic Information Systems*. Association for Computing Machinery. doi:10.1145/3841645.3843430 To appear.

[23] A. Savitzky and M. J. E. Golay. 1964. Smoothing and Differentiation of Data by Simplified Least Squares Procedures. *Analytical Chemistry* 36, 8 (1964), 1627–1639. doi:10.1021/ac60214a047

[24] Chris Stanford, Suman Adari, Xishun Liao, Yueshuai He, Qinhua Jiang, Chenchen Kuai, Jiaqi Ma, Emmanuel Tung, Yinlong Qian, Lingyi Zhao, et al. 2024. Numosim: A synthetic mobility dataset with anomaly detection benchmarks. In *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Geospatial Anomaly Detection*. 68–78.

[25] George E Uhlenbeck and Leonard S Ornstein. 1930. On the theory of the Brownian motion. *Physical review* 36, 5 (1930), 823.

[26] Valhalla Project. 2026. Meili – Valhalla map matching. https://valhalla.github.io/valhalla/meili/. Accessed: 2026-03-07.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[28] Paul A. Zandbergen and Sean J. Barbeau. 2011. Positional Accuracy of Assisted GPS Data from High-Sensitivity GPS-enabled Mobile Phones. *The Journal of Navigation* 64, 3 (2011), 381–399. doi:10.1017/S0373463311000051

[29] Yu Zheng. 2015. Trajectory data mining: an overview. *ACM Transactions on Intelligent Systems and Technology (TIST)* 6, 3 (2015), 1–41.

[30] Yu Zheng, Licia Capra, Ouri Wolfson, and Hai Yang. 2014. Urban computing: concepts, methodologies, and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* 5, 3 (2014), 1–55.

[31] Lei Zhu, Jacob R Holden, and Jeffrey D Gonder. 2017. Trajectory segmentation map-matching approach for large-scale, high-resolution GPS data. *Transportation Research Record* 2645, 1 (2017), 67–75.

[32] Yuanshao Zhu, Yongchao Ye, Shiyao Zhang, Xiangyu Zhao, and James Yu. 2023. DiffTraj: Generating GPS Trajectory with Diffusion Probabilistic Model. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 65168–65188. https://proceedings.neurips.cc/paper_files/paper/2023/file/cd9b4a28fb9eebe0430c3312a4898a41-Paper-Conference.pdf

[33] Yuanshao Zhu, James Jianqiao Yu, Xiangyu Zhao, Qidong Liu, Yongchao Ye, Wei Chen, Zijian Zhang, Xuetao Wei, and Yuxuan Liang. 2024. ControlTraj: Controllable Trajectory Generation with Topology-Constrained Diffusion Model. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 4676–4687. doi:10.1145/3637528.3671866